AIML ZG511: Deep Neural Networks

Master Practice Problem Set (Combined Expanded)

Final Exam Revision - Complete Collection

Contents

Preface

This document contains the **complete union** of all practice questions provided. No questions have been removed. Duplicate topics (like Perceptron iterations) are treated as separate practice scenarios (Scenario A and Scenario B) to provide maximum practice. Solutions are expanded to show every step of the calculation.

1 Module 1: Perceptron & Linear Foundations

Q1. Perceptron Numerical Iteration (Scenario A)

Setup:

- Weights: $w = [w_0, w_1, w_2]^T = [0.5, 0.3, -0.4]^T$. (Bias is w_0).
- Data: Input $x = [1, 2]^T$. True Label $y = 1$. (Note: Bias input $x_0 = 1$).
- Settings: $\eta = 0.1$. Activation: Step Function (1 if $z \geq 0$, else 0).

Task: Compute z, $\hat{y}$, and update weights if necessary.

Detailed Solution & Step-by-Step Logic

Step 1: Construct Input Vector
We must prepend the bias input $x_0 = 1$ to the feature vector.

$$\mathbf{x} = [1, 1, 2]^T$$

Step 2: Calculate Weighted Sum (z)

$$z = \mathbf{w}^T\mathbf{x} = (w_0 \cdot x_0) + (w_1 \cdot x_1) + (w_2 \cdot x_2)$$
$$z = (0.5 \cdot 1) + (0.3 \cdot 1) + (-0.4 \cdot 2)$$
$$z = 0.5 + 0.3 - 0.8$$
$$z = 0.0$$

Step 3: Apply Activation ($\hat{y}$)
The condition is $z \geq 0$.

$$0.0 \geq 0 \implies \hat{\mathbf{y}} = \mathbf{1}$$

Step 4: Check Error & Update

$$\text{Error} = y - \hat{y} = 1 - 1 = 0$$

Since the error is 0, the prediction is correct.

Result: No weight update is required.

Weights remain: $[0.5, 0.3, -0.4]^T$.

Q2. Perceptron Numerical Iteration (Scenario B)

Setup:

- Weights: $w = [0.5, -0.5, 0.5]^T$ (Includes bias).
- Data: Input $x_{data} = [2, 4]$. True label $y = 0$. (Bias input $x_0 = 1$).
- Settings: $\eta = 0.2$. Activation: Step Function.

Task: Perform one training step.

Detailed Solution & Step-by-Step Logic

Step 1: Weighted Sum (z)

$$z = (0.5 \cdot 1) + (-0.5 \cdot 2) + (0.5 \cdot 4)$$
$$z = 0.5 - 1.0 + 2.0 = 1.5$$

Step 2: Prediction ($\hat{y}$)
Since $1.5 \geq 0$, the prediction is $\hat{y} = 1$.
Step 3: Error Calculation

$$\text{Error} = \text{Target} - \text{Prediction} = 0 - 1 = -1$$

Step 4: Weight Update
Formula: $w_{new} = w_{old} + \eta \cdot \text{Error} \cdot \mathbf{x}$

$$\Delta w = 0.2 \cdot (-1) \cdot \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} = \begin{bmatrix} -0.2 \\ -0.4 \\ -0.8 \end{bmatrix}$$

$$w_{new} = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.5 \end{bmatrix} + \begin{bmatrix} -0.2 \\ -0.4 \\ -0.8 \end{bmatrix} = \begin{bmatrix} \mathbf{0.3} \\ \mathbf{-0.9} \\ \mathbf{-0.3} \end{bmatrix}$$

Q3. Perceptron: Geometric Interpretation

A trained Perceptron has weights $w = [w_0, w_1, w_2] = [-2, 1, 1]$.

1. Write the decision boundary equation in the form $x_2 = mx_1 + c$.
2. Classify the point $A = (3, 0)$.

Detailed Solution & Step-by-Step Logic

1. Equation of the Line
The boundary is where $z = 0$:

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

$$-2 + 1x_1 + 1x_2 = 0$$

Rearrange to isolate x_2:

$$x_2 = -x_1 + 2$$

(Slope $m = -1$, Intercept $c = 2$).

2. Classification of Point (3,0)
Calculate z:

$$z = -2 + (1 \cdot 3) + (1 \cdot 0) = -2 + 3 = +1$$

Since $z(+1) \geq 0$, the Class is **1** (Positive Class).

Q4. Interpretability & Feature Engineering

A spam classifier has weights $w = [0.2, 0.8, 0.9, -0.5]^T$ corresponding to $[Bias, \text{SuspiciousWords}, \text{Links}, \text{Leng}$

1. Which feature most strongly indicates spam?
2. If accuracy stalls at 75%, what does this imply about geometry?

Detailed Solution & Step-by-Step Logic

1. Strongest Feature: Links.
Reason: It has the largest positive weight (0.9). This means an increase in links contributes most heavily to the positive sum (Class "Spam").
2. Geometry Implications.
It implies the data is **Not Linearly Separable**. A perceptron defines a flat hyperplane. If accuracy stalls, the Spam and Not-Spam clusters overlap in a way a straight line cannot divide.

2 Module 2: Linear Regression & Gradient Descent

Q5. Batch Gradient Descent (Numerical)

Model: $\hat{y} = w_0 + w_1(\text{Area})$. **Current State:** $w_0 = 50, w_1 = 8$. $\eta = 0.01$. **Data:** Sample A $(10, 150)$ and Sample B $(20, 250)$. **Loss:** MSE (Gradient factor $2/N$). Perform one iteration of Batch GD.

Detailed Solution & Step-by-Step Logic

Step 1: Predictions
Sample A: $\hat{y}_A = 50 + 8(10) = 130$.
Sample B: $\hat{y}_B = 50 + 8(20) = 210$.
Step 2: Errors $(y_{pred} - y_{true})$
$e_A = 130 - 150 = -20$.
$e_B = 210 - 250 = -40$.
Step 3: Compute Gradients
Formula: $\frac{\partial L}{\partial w} = \frac{2}{N} \sum(\text{error}) \cdot \text{input}$. (Here $N = 2$).

$$\nabla w_0 = \frac{2}{2}[(-20 \cdot 1) + (-40 \cdot 1)] = -60$$

$$\nabla w_1 = \frac{2}{2}[(-20 \cdot 10) + (-40 \cdot 20)] = -200 - 800 = -1000$$

Step 4: Update Weights
$w_0^{new} = 50 - 0.01(-60) = 50 + 0.6 = \mathbf{50.6}$.
$w_1^{new} = 8 - 0.01(-1000) = 8 + 10.0 = \mathbf{18.0}$.

Q6. Code Completion: Vectorized Linear Regression

Fill in the blanks to implement vectorized Batch GD.

```python
import numpy as np
def train_linear(X, y, lr=0.01, epochs=100):
    # X shape: (N, d), y shape: (N, 1)
    N, d = X.shape
    weights = np.zeros((d, 1))

    for i in range(epochs):
        # Blank 1: Predictions (Vectorized Matrix Mult)
        y_pred = X @ weights

        # Blank 2: Error Vector
        error = y_pred - y

        # Blank 3: Gradient Formula (MSE 2/N)
        # Transpose X to align dimensions: (d, N) @ (N, 1) -> (d, 1)
        gradient = (2/N) * (X.T @ error)

        # Blank 4: Update
        weights = weights - lr * gradient

    return weights
```

Q7. Feature Scaling Scenario

Model A is trained on raw Area (400-2000 sq ft). Model B is trained on Normalized Area (Z-score).

- **Which trains faster?** Model B.
- **Why?** In Model A, the error surface is an elongated valley (elliptical contours) because a small change in w_{area} affects the loss massively compared to w_{bias}. This forces a very small learning rate to avoid divergence. Model B has a spherical error surface, allowing a higher learning rate and direct path to the minimum.

3 Module 3: Logistic Regression & Classification

Q8. Logistic Step Calculation

Weights $w = [0, 0.6, 0.8]$ (Bias, Credit, Income). Input $x = [0.7, 0.5]$. True $y = 1$. $\eta = 0.1$. Perform one update for w_1 (Credit).

Detailed Solution & Step-by-Step Logic

Step 1: Weighted Sum (z)
$z = (0 \cdot 1) + (0.6 \cdot 0.7) + (0.8 \cdot 0.5) = 0 + 0.42 + 0.40 = 0.82$.
Step 2: Sigmoid Activation ($\hat{y}$)

$$\hat{y} = \frac{1}{1 + e^{-0.82}} \approx \frac{1}{1 + 0.4404} \approx 0.694$$

Step 3: Gradient for w_1
Gradient term $= (\hat{y} - y) \cdot x_1$.

$$\nabla w_1 = (0.694 - 1) \cdot 0.7 = (-0.306) \cdot 0.7 = -0.2142$$

Step 4: Update

$$w_1^{new} = 0.6 - 0.1(-0.2142) = 0.6 + 0.02142 = \mathbf{0.6214}$$

Q9. Logistic Loss (Binary Cross-Entropy)

A logistic neuron calculates $z = -2.0$. True label $y = 1$. Calculate the loss.

Detailed Solution & Step-by-Step Logic

Step 1: Probability

$$\hat{y} = \frac{1}{1 + e^{-(-2)}} = \frac{1}{1 + 7.389} \approx 0.119$$

The model is 88% sure it is Class 0, but it is actually Class 1.
Step 2: Cross-Entropy Loss

$$L = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

Since $y = 1$, the second term vanishes:

$$L = -1 \cdot \ln(0.119) \approx -(-2.128) = \mathbf{2.128}$$

Q10. Neural Architecture: Parameter Counting

Designing an MLP for 28×28 flattened images. Structure: Input(784) $\rightarrow$ H1(200) $\rightarrow$ H2(100) $\rightarrow$ Output(10).

Detailed Solution & Step-by-Step Logic

Formula: (Input $\times$ Neurons) + Biases.
Layer 1: $(784 \times 200) + 200 = 156,800 + 200 = 157,000$.
Layer 2: $(200 \times 100) + 100 = 20,000 + 100 = 20,100$.
Output: $(100 \times 10) + 10 = 1,000 + 10 = 1,010$.
Total: $157,000 + 20,100 + 1,010 = \mathbf{178,110}$ parameters.

Q11. Activation Functions Conceptual

Why shift from Sigmoid/Tanh to ReLU for deep networks?

Detailed Solution & Step-by-Step Logic

Vanishing Gradient Problem: Sigmoid derivatives have a max value of 0.25. Tanh maxes at 1.0 but decays exponentially fast away from zero. When training deep networks,

backpropagation multiplies these derivatives at every layer.

$$0.25 \times 0.25 \times 0.25 \cdots \approx 0$$

The gradient vanishes before reaching early layers. **ReLU Advantage:** For $z > 0$, the derivative is exactly 1. Multiplying by 1 preserves the gradient magnitude, allowing deep networks to learn.

4 Module 4: Deep Learning - Forward Backward

Q12. Matrix Forward Propagation

Input $x = [0.5, 0.8]^T$. $W^{[1]} = \begin{pmatrix} 1.0 & 0.5 \\ -0.5 & 1.5 \end{pmatrix}, b^{[1]} = \begin{pmatrix} 0.2 \\ -0.3 \end{pmatrix}$. Activation: ReLU (Hidden), Sigmoid (Output). Output Layer: $W^{[2]} = [1.2, 0.8]^T, b^{[2]} = 0.1$.

Detailed Solution & Step-by-Step Logic

1. Hidden Layer Linear Step

$$Z^{[1]} = W^{[1]}x + b^{[1]}$$

Row 1: $(1.0)(0.5) + (0.5)(0.8) + 0.2 = 0.5 + 0.4 + 0.2 = 1.1$.
Row 2: $(-0.5)(0.5) + (1.5)(0.8) - 0.3 = -0.25 + 1.2 - 0.3 = 0.65$.
2. Hidden Layer Activation
$A^{[1]} = \text{ReLU}([1.1, 0.65]) = [1.1, 0.65]^T$.
3. Output Layer
$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$ (Dot product)
$Z^{[2]} = (1.2)(1.1) + (0.8)(0.65) + 0.1$
$Z^{[2]} = 1.32 + 0.52 + 0.1 = 1.94$.
$\hat{y} = \sigma(1.94) = \frac{1}{1+e^{-1.94}} \approx \mathbf{0.874}$.

Q13. Computational Graph Chain Rule

Function $J = (a + 2b)^2$. Let $u = a + 2b$. Given $a = 3, b = 1$. Compute $\frac{\partial J}{\partial b}$.

Detailed Solution & Step-by-Step Logic

Forward Pass: $u = 3 + 2(1) = 5$. $J = 5^2 = 25$.
Backward Pass: Path: $b \to u \to J$. 1. $\frac{\partial J}{\partial u} = 2u = 2(5) = 10$. 2. $\frac{\partial u}{\partial b} = 2$. 3. $\frac{\partial J}{\partial b} = 10 \times 2 = \mathbf{20}$.

Q14. Numerical Backpropagation (Comprehensive)

Setup: Inputs $i_1 = 0.5, i_2 = 1.0$. Target $t = 0.8$. Hidden: $w_1 = 0.2, w_2 = -0.4, b_h = 0.1$. (Using Sigmoid). Output: $w_{out} = 0.6, b_{out} = -0.2$. (Using Sigmoid). $\eta = 0.5$. Loss: Squared Error $\frac{1}{2}(t - y)^2$.

Detailed Solution & Step-by-Step Logic

Phase A: Forward Pass
$net_h = (0.2)(0.5) + (-0.4)(1) + 0.1 = 0.1 - 0.4 + 0.1 = -0.2$.
$out_h = \sigma(-0.2) = 0.450$.
$net_o = (0.6)(0.450) - 0.2 = 0.27 - 0.2 = 0.07$.
$out_o = \sigma(0.07) = 0.517$.
Phase B: Gradient at Output
$\frac{\partial E}{\partial out_o} = -(0.8 - 0.517) = -0.283$.
$\frac{\partial out_o}{\partial net_o} = out_o(1 - out_o) = 0.517(0.483) = 0.2497$.
$\delta_o = -0.283 \times 0.2497 = -0.07067$.
Phase C: Gradient at Hidden Weight (w_{i1})
We need error signal at hidden node δ_h:

$$\delta_h = (\delta_o \cdot w_{out}) \cdot [out_h(1 - out_h)]$$

$$\delta_h = (-0.07067 \cdot 0.6) \cdot [0.450 \cdot 0.550]$$

$$\delta_h = (-0.0424) \cdot (0.2475) = -0.0105$$

Gradient for w_{i1}:

$$\frac{\partial E}{\partial w_{i1}} = \delta_h \cdot i_1 = -0.0105 \cdot 0.5 = -0.00525$$

Phase D: Update
$w_{i1}^{new} = 0.2 - 0.5(-0.00525) = \mathbf{0.2026}$.

Q15. PyTorch Implementation Code

Complete the standard PyTorch training loop.

```
# Assume model, optimizer, criterion defined
for epoch in range(epochs):
    # 1. Clear accumulated gradients
    optimizer.zero_grad()

    # 2. Forward pass
    outputs = model(inputs)
    loss = criterion(outputs, labels)

    # 3. Backward pass (Autograd)
    loss.backward()

    # 4. Parameter update
    optimizer.step()
```

5 Module 5: Metrics, Design & Optimization

Q16. Cost Analysis (Imbalance Scenario A)

9500 Legit, 500 Fraud. Cost(Miss)=100, Cost(False Alarm)=2.

- Model A: Misses 200 frauds, 50 False Alarms.

- Model B: Misses 125 frauds, 150 False Alarms.

Detailed Solution & Step-by-Step Logic

Model A Cost: $(200 \times 100) + (50 \times 2) = 20,000 + 100 = 20,100$.
Model B Cost: $(125 \times 100) + (150 \times 2) = 12,500 + 300 = 12,800$.
Conclusion: Model B is superior despite higher false alarms.

Q17. Accuracy Paradox (Imbalance Scenario B)

99,000 Legit, 1,000 Fraud. Model predicts "Legit" for everyone.

- Accuracy: $99,000/100,000 = 99\%$.
- Recall (Fraud): $0/1000 = 0\%$.

Detailed Solution & Step-by-Step Logic

Insight: Accuracy is dangerous here. The model is functionally useless (caught 0 frauds) despite 99% accuracy.

Q18. Softmax Calculation

Logits $z = [2.0, 1.0, 0.5]$.

1. Calculate $P(\text{Class 1})$.
2. Calculate Cross-Entropy if Class 1 is true.

Detailed Solution & Step-by-Step Logic

Exponentials: $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^{0.5} \approx 1.65$.
Sum: 11.76.
$P(\text{Class 1}) = 7.39/11.76 \approx \mathbf{0.628}$.
Loss $= -\ln(0.628) \approx \mathbf{0.465}$.

Q19. Confusion Matrix Diagnostics

True Birds: 300. Predicted: Bird (250), Dog (40), Cat (10).

Detailed Solution & Step-by-Step Logic

Recall $= 250/300 = 83.3\%$. **Diagnosis:** High confusion with Dog (40) suggests the model isn't learning features specific enough to distinguish these two (likely relying on background/texture rather than shape).

Q20. Learning Curve Diagnosis

Plot shows Training Loss decreasing smoothly, but Validation Loss increasing after epoch 3.

Detailed Solution & Step-by-Step Logic

Diagnosis: Overfitting. The model is memorizing noise in the training set. **Fixes:** 1. Regularization (Dropout, L2). 2. Early Stopping (Stop at epoch 3). 3. Data Augmentation.

Q21. Optimization Concepts (SGD, Momentum, Dropout)

Detailed Solution & Step-by-Step Logic

- **SGD vs Batch:** Batch is too slow/memory-heavy for large data. SGD is fast but noisy.
- **Momentum:** Adds a velocity term to breeze through flat areas and dampen oscillation in ravines.
- **L2 Regularization:** Mathematically results in weight decay ($w = w - \eta\lambda w$).
- **Dropout:** Randomly kills neurons during training to prevent co-adaptation. Must scale weights at test time to match expected magnitude.

Q22. Design Scenarios

Detailed Solution & Step-by-Step Logic

Mobile Sentiment Analysis: Use a small embedding and a bottleneck MLP (e.g., 1000 $\rightarrow$ 64) for speed/memory. **House Prices:** Output Linear/ReLU. Loss: MSE. **Image Digits:** Input 784 (or 28x28 CNN). Output 10 (Softmax). Loss: Categorical Cross-Entropy.

6 Rapid Fire Conceptual Checks

1. **XOR Problem:** Cannot be solved by a single perceptron (not linearly separable).
2. **Zero Initialization:** Bad. Hidden neurons will remain symmetrical and never learn different features.
3. **High Learning Rate:** Causes divergence or oscillation.
4. **Batch Norm:** Normalizes layer inputs, allowing faster training and higher learning rates.
5. **Bias vs Variance:** High Training Loss = High Bias (Underfitting). High Val Loss + Low Train Loss = High Variance (Overfitting).
6. **Softmax Property:** If one class probability rises, others must fall (Sum is 1).